

Metriche e Qualità del Software

Introduzione

La misura nel software

- ◆ Misura di attributi di processo
- ◆ Misura di attributi di prodotto
 - Attributi interni
 - Attributi esterni
- ◆ Misura di attributi delle risorse

Misura di attributi di processo

- ◆ Misurare le caratteristiche del processo di sviluppo è cruciale
 - La qualità del processo ha un impatto rilevante sulla qualità del prodotto
 - Diversi attributi esterni del software (es. affidabilità e manutenibilità) possono essere misurati indirettamente attraverso misura di attributi di processo (es. numero di malfunzionamenti individuati durante il beta test, sforzo della fase di manutenzione)

Misura di attributi di processo: definizioni

◆ Errore

- La causa di un difetto
- Esempio: non aver compreso i requisiti

◆ Difetto

- L' errore nel codice o nel documento di specifica/progetto

◆ Malfunzionamento

- Il risultato esterno e visibile di un difetto
- Esempio: l' applicazione fornisce risultati errati in una particolare situazione

Attributi di processo: cosa misurare

- ◆ Posizione
 - Dove è una certa entità, dove si verifica un certo evento
- ◆ Tempo
 - Quando si verifica un certo evento
- ◆ Effetto
 - Che cosa si può osservare
- ◆ Manifestazione
 - Come si manifesta un certo evento
- ◆ Causa
 - Qual è la causa di un certo evento
- ◆ Costo
 - Quanto costa una determinata entità
- ◆ Conteggio
 - Quante entità o eventi sono stati osservati

Malfunzionamenti (1)

◆ Posizione

- Un codice che identifichi univocamente la configurazione per la quale si sia osservato il malfunzionamento (scala nominale)

◆ Tempo

- Tempo effettivo (data-ora) nel quale si è osservato il malfunzionamento (scala intervallo)
- Tempo di esecuzione dopo il quale si è rilevato il malfunzionamento (scala rateo)

◆ Effetti

- Espressi attraverso una classificazione (scala ordinale) che descriva gli effetti osservati (es. crash del sistema, crash dell' applicazione, perdita di dati, ecc.)

Malfunzionamenti (2)

◆ Manifestazione

- In quali condizioni si è presentato il malfunzionamento
- Si usa una classificazione (scala nominale)

◆ Causa

- Si usa una classificazione (scala ordinale)
- Es. Errore di progetto, errore utente, errore hardware, ecc.

◆ Costo

- Costo per l'utente: misurato classificando la severità del malfunzionamento (scala ordinale) o stimando il costo effettivo (scala rateo)
- Costo per il produttore: costo di individuazione e rimozione del difetto che ha portato al malfunzionamento (scala rateo)

◆ Conteggio

- Quanti malfunzionamenti, per classe di severità (scala assoluta)

Difetti (1)

◆ Posizione

- Modulo e versione che presenta il difetto (scala nominale)

◆ Tempo

- Tempo effettivo di introduzione del difetto (scala intervallo)
- Tempo effettivo di individuazione del difetto (scala intervallo)

◆ Effetti

- Tipologia di malfunzionamento derivante dal difetto (scala ordinale)

◆ Manifestazione

- Come è stato individuato il difetto (scala nominale)
- Classificazione del difetto (scala ordinale)

Difetti (2)

◆ Causa

- Classificazione dell' errore che ha portato al difetto (scala nominale)

◆ Costo

- Costo per l' utente e per il produttore

◆ Conteggio

- Numero di difetti per modulo, numero di difetti individuati durante una certa fase del processo, ecc. (scale assolute)

Modifiche

- ◆ Tenere traccia delle modifiche apportate al prodotto durante il processo è una parte fondamentale del controllo delle configurazioni (“change control”)
- ◆ Locazione
 - Modulo e versione modificata
- ◆ Tempo
 - Tempo effettivo di effettuazione della modifica
- ◆ Causa
 - Classificazione della causa della modifica
- ◆ Costo
 - costo di implementazione della modifica
- ◆ Conteggio
 - Numero di modifiche introdotte in un dato periodo di tempo

La misura degli attributi interni dei prodotti (1)

- ◆ Negli ultimi trent'anni l'ingegneria del software ha prodotto tecniche e metodologie per lo sviluppo di software con certe caratteristiche “interne” (modularità, riuso, accoppiamento, coesione, ...)
 - Tali caratteristiche devono essere misurate
- ◆ Gli attributi esterni che maggiormente influiscono sulla qualità del prodotto dipendono, in varia misura, dagli attributi interni
- ◆ Gli attributi interni ci consentono di definire, con una ragionevole precisione, l'importante attributo di “complessità” del prodotto
 - Anche se è difficile indicare con un numero la complessità di un prodotto, diversi fattori interni contribuiscono a definirla

La misura degli attributi interni dei prodotti (2)

- ◆ Gli attributi interni possono essere misurati fin dalle prime fasi del ciclo di vita
 - Per stimare attributi del prodotto finito
 - Per valutare la “qualità” dei documenti di specifica e progetto
- ◆ La misura degli attributi interni del prodotto permette di valutare l'efficacia dei metodi usati durante lo sviluppo

La dimensione (1)

- ◆ La “dimensione” di un prodotto dovrebbe essere uno degli attributi più semplici da misurare
- ◆ Purtroppo le misure proposte sono inficiate da un problema di fondo:
 - La dimensione viene vista solo come uno strumento per stimare altre caratteristiche (sforzo di sviluppo, costo)
 - » Questo crea una confusione sulla nozione di “dimensione”
 - » E' comune che certe misure di dimensione vengano scartate perché “non riflettono adeguatamente lo sforzo di produzione” oppure “non riflettono la produttività perché non tengono conto della complessità”

La dimensione (2)

- ◆ Una corretta misura della dimensione deve prescindere da tali caratteristiche
 - Occorre misurare solo ciò che empiricamente intendiamo con “dimensione”
- ◆ Esistono diverse forme di “dimensione”
 - Lunghezza
 - Funzionalità fornita
 - Complessità del problema che il prodotto deve risolvere

Lunghezza (1)

- ◆ La più nota dimensione di lunghezza per i sorgenti è il numero di linee di codice
 - Distinguiamo tra
 - » NCLOC: linee di codice senza contare i commenti
 - » LOC: linee di codice (commenti inclusi)
 - » CLOC: linee di commenti
 - » $LOC = NCLOC + CLOC$
- ◆ Ciò ci permette di definire alcune misure ausiliarie come la “densità dei commenti”:
- ◆ CLOC/LOC

Lunghezza (2)

- ◆ Altre misure di lunghezza per i sorgenti
 - Dimensione in byte
 - Dimensione in caratteri
- ◆ Misura della lunghezza di documenti di specifica e progetto
 - Tali documenti sono espressi con linguaggi diversi (alcuni addirittura grafici)
 - Possibili misure:
 - » Numero di “pagine”
 - » Dimensione in byte

Lunghezza: sistemi predittivi

- ◆ La lunghezza del codice viene utilizzata per stimare importanti attributi di processo quali sforzo e costo
 - Diventa fondamentale stimare la lunghezza del codice durante le prime fasi del ciclo di vita
- ◆ Metodi di stima fondati sulla conoscenza del “rateo di espansione specifica-codice”
- ◆ $\alpha = \text{lung. specifica} / \text{lung. codice}$
- ◆ Il rateo di espansione deve essere calcolato a partire da una analisi di progetti analoghi

Funzionalità

- ◆ La più diffusa misura della funzionalità sono i punti funzione (Function Points)
- ◆ Difetti:
 - Misura fortemente soggettiva
 - Misura dipendente dal livello di astrazione al quale ci si pone
 - Si cerca di misurare non solo le funzionalità fornite ma anche la complessità di tali funzionalità (fattori correttivi)
 - ...

Complessità del problema (1)

- ◆ Un programma risolve un problema
 - La complessità di tale problema è una caratteristica “interna” del programma
- ◆ Possiamo definire tale attributo come la quantità di risorse necessarie a risolvere il problema
 - Risorse: tempo, spazio
- ◆ La complessità dei problemi si misura attraverso la funzione O
 - Data una funzione $f(n)$ con n misura della dimensione del problema (es. lunghezza di una lista da ordinare) $O(f(n))$ è l'insieme delle funzioni $g(n)$ tali che:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$$

Complessità del problema (2)

- ◆ Informalmente dire che un problema ha complessità $O(f(n))$ equivale a dire che ogni algoritmo per la sua soluzione appartiene all'insieme $O(f(n))$
- ◆ Possiamo usare l'inclusione insiemistica come relazione numerica che descriva la relazione empirica “più complesso”

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(c^n) \subset O(n!)$$

$$\forall i < j (O(n^i) \subset O(n^j))$$

$$\forall i (O(n^i) \subset O(c^n))$$

Complessità del problema (3)

◆ Esempio:

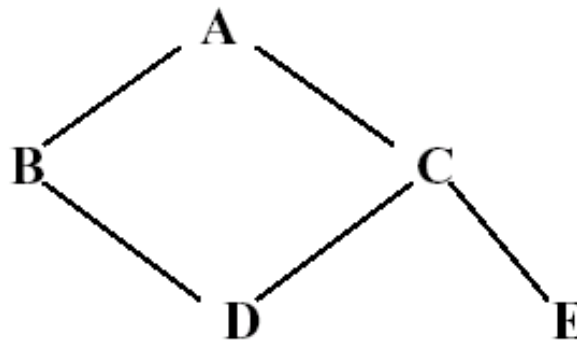
- Il problema della ricerca di un elemento in una lista ordinata è $O(\log n)$
- Il problema di ordinare una lista è $O(n \log n)$
- Il primo problema è meno complesso del secondo

Misure di modularizzazione (1)

- ◆ Modulo: una sequenza di istruzioni o procedure incluse all'interno di una coppia di “delimitatori” che abbia un identificatore unico
- ◆ La definizione è volutamente vaga per includere diversi concetti in funzione del linguaggio di programmazione adottato
 - Insiemi di procedure C
 - Package ADA
 - Classi o insiemi di classi C++
 - Classi o package Java
- ◆ La modularizzazione è l'attributo di un programma che descrive il modo con il quale è stato suddiviso in moduli

Misure di modularizzazione (2)

- ◆ Possibili misure del livello di modularizzazione
 - Lunghezza media di un modulo
 - Num. moduli/Num. procedure
 - Num. moduli/Num. variabili
- ◆ Il grafo delle dipendenze tra moduli



Misure di modularizzazione (3)

- ◆ Misure di modularizzazione basate sull'osservazione del grafo delle dipendenze
 - Dimensione: numero di nodi e numero di archi
 - Profondità: lunghezza del più lungo cammino dalla radice ad una delle foglie
 - Densità di connessione: rapporto tra numero di nodi e numero di archi

Misure di modularizzazione: tree impurity

- ◆ Informalmente definita come la “distanza” tra il grafo delle dipendenze ed un albero avente lo stesso numero di nodi
- ◆ Tale misura dovrebbe essere mantenuta “bassa”
 - Un grafo delle dipendenze che sia molto “ingarbugliato” riflette un sistema nel quale per comprendere ogni modulo bisogna comprenderne molti altri
- ◆ Si può verificare che una misura della tree impurity deve soddisfare un insieme di relazioni empiriche tali per cui una possibile espressione formale di tale misura è data dalla formula:

$$M(G) = \frac{2(e - n + 1)}{(n - 1)(n - 2)}$$

dove e è il numero di archi del grafo G , ed n è il numero di nodi

Riuso

- ◆ Distinguiamo tra:
 - Riuso pubblico, definito come il riuso di codice prodotto da altri o in altri progetti
 - Riuso privato, definito come il riuso di codice all' interno dello stesso progetto
- ◆ Siamo interessati a misurare il primo tipo di riuso
 - Il grafo delle dipendenze ci dice che un certo modulo ne usa altri
 - » Non ci dice quante volte li usi (una sola chiamata o molte?)
 - » Una misura del riuso ottenuta a partire dal solo grafo delle dipendenze che soddisfa le relazioni empiriche di tale misura è:
$$r(G)=e-n+1$$
 - Misure più significative dovrebbero tenere conto del numero di chiamate e della dimensione dei moduli riutilizzati

Accoppiamento (1)

- ◆ L' accoppiamento tra due moduli può essere
 - Nullo: non c' è alcuna relazione tra i due moduli
 - Relativo ai dati: se i due moduli “comunicano” attraverso dati scambiati come parametri o valori di ritorno di mutue invocazioni
 - Relativo ai tipi: se i due moduli usano lo stesso tipo di dato (record o struct) come parametro
 - Relativo al controllo: se un modulo passa all' altro come parametro un' informazione che influisce sul flusso di controllo di questo
 - Relativo all' ambiente globale: se i due moduli comunicano attraverso variabili globali
 - Relativo al contenuto: se un modulo modifica variabili di un altro o “vi salta dentro”

Accoppiamento (2)

- ◆ Tale classificazione fornisce la base per una misura (scala ordinale) dell' accoppiamento tra due moduli
- ◆ L' accoppiamento complessivo di un sistema può essere misurato come valor medio dell' accoppiamento dei suoi moduli quando ad ogni tipo di accoppiamento si sia dato un valore numerico (da 0 a 5)

Coesione (1)

- ◆ La coesione è un attributo interno al singolo modulo
- ◆ Una possibile classifica (in ordine decrescente) :
 - Funzionale: il modulo realizza un' unica funzione concettuale
 - Sequenziale: Il modulo racchiude più funzionalità ma tutte invocate in sequenza dal sistema
 - A livello di dati: Il modulo racchiude più funzioni che operano sugli stessi dati
 - Procedurale: Il modulo racchiude più funzioni tutte collegate ad una procedura generale del software
 - Temporale: Il modulo racchiude più funzioni che vengono invocate dal software nello stesso “periodo di tempo”
 - Logico: Il modulo racchiude più funzioni aventi una certa relazione logica
 - Accidentale: Il modulo racchiude più funzioni senza alcuna relazione tra loro

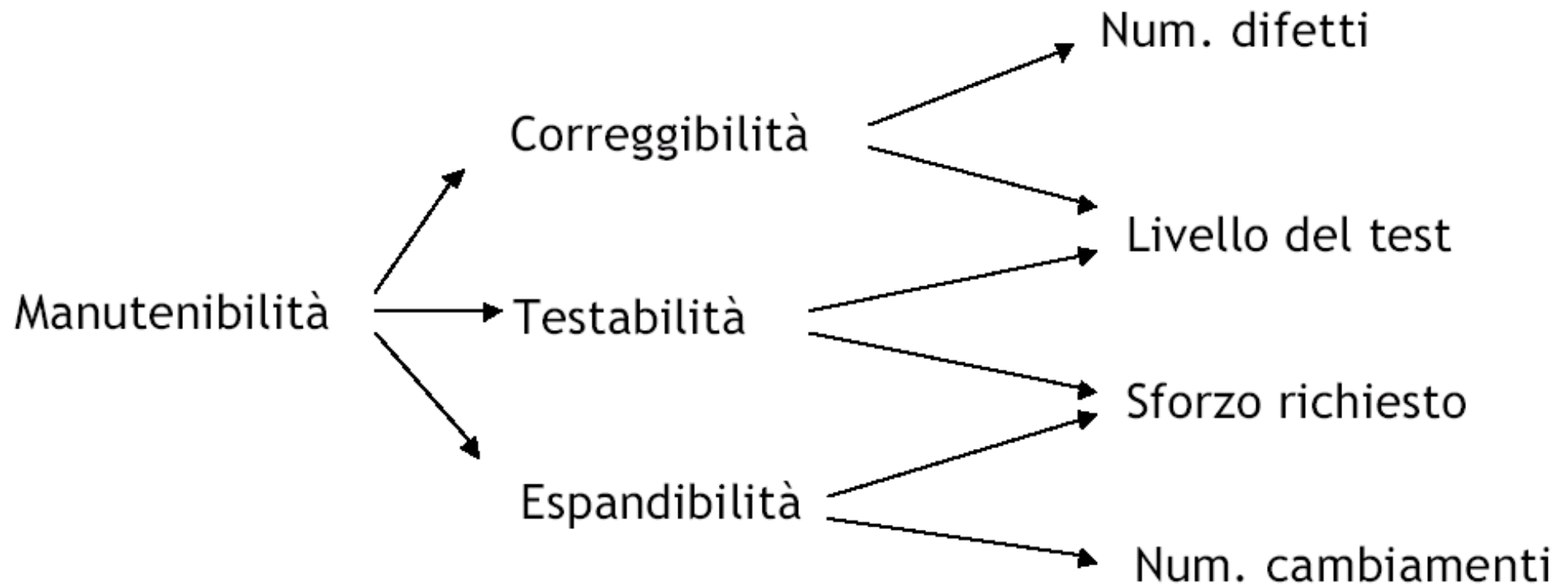
La misura degli attributi esterni di prodotto

- ◆ Gli attributi esterni del prodotto sono definiti in relazione all' ambiente esterno
 - In particolare all' utente del prodotto
- ◆ Sono spesso definiti “attributi di qualità”
- ◆ Tale qualità è caratterizzata da diversi aspetti. I principali:
 - Affidabilità
 - Usabilità
 - Manutenibilità

I modelli di qualità

- ◆ La misura degli attributi di qualità è una attività complessa
 - Bisogna decidere da quali attributi effettivamente misurabili dipendano gli attributi di qualità esterni
 - Non esiste un accordo
 - Si può definire un “modello di qualità”
 - » Mette in relazione i fattori di qualità (es. manutenibilità), i criteri di qualità ovvero gli attributi misurabili da cui questi dipendono (es. correggibilità, espandibilità), e le metriche che effettivamente ci danno una misura dei criteri

I modelli di qualità: un esempio



Qualità: una definizione ridotta

- ◆ In parecchi casi ci accontentiamo di una visione ridotta del concetto di qualità:
 - Qualità come assenza di difetti
- ◆ Una semplice metrica che misuri tale “forma” di qualità
 - Numero di difetti noti/dimensione

Affidabilità (1)

- ◆ L'affidabilità di un prodotto è data dal tempo medio tra due malfunzionamenti (MTBF)
- ◆ Questa dipende solo in parte dal numero di difetti presenti
 - Certi difetti potrebbero non dare mai luogo a malfunzionamenti
 - Certi difetti si manifestano sotto forma di malfunzionamenti molto frequenti
- ◆ L'affidabilità può essere misurata solo a posteriori
 - Deve essere stimata nota la storia del funzionamento del sistema fino ad oggi

Affidabilità (2)

- ◆ Esistono diversi modelli predittivi per la stima dell'affidabilità di un sistema
- ◆ I due tipi di errori a cui tali modelli vanno incontro sono “bias” e “noise”
 - In generale l'affidabilità cresce nel tempo ma è soggetta a fluttuazioni (la correzione di difetti talvolta ne introduce di nuovi)
 - Con il termine “bias” intendiamo l'errore nel sottostimare o sovrastimare l'affidabilità del sistema
 - Con il termine “noise” intendiamo l'errore nello stimare le fluttuazioni dell'affidabilità
- ◆ Esistono diversi modelli statistici per valutare l'errore di un modello di stima dell'affidabilità
 - Non entriamo nei dettagli...

Usabilità

- ◆ L'usabilità potrebbe essere definita come:
 - La probabilità che l'operatore non incontri problemi di interfaccia durante un certo periodo
- ◆ Problema
 - Collezionare dati per un tale attributo si rivela quasi impossibile
- ◆ Soluzione
 - Possiamo usare la misura del tempo necessario ad imparare ad usare il sistema come misura approssimata di usabilità
 - Non tiene conto della “gradevolezza” del sistema

Manutenibilità (1)

- ◆ La manutenibilità è definita come la facilità con la quale il prodotto può essere compreso, corretto, adattato, migliorato
- ◆ Anche questo fattore di qualità non può essere misurato direttamente
- ◆ Misure indirette
 - Tempo medio per effettuare una modifica
 - Numero di malfunzionamenti irrisolti
 - Tempo speso nel tentativo di risolvere malfunzionamenti non ancora risolti
 - Percentuale di cambiamenti che introducono nuovi difetti
 - Numero di moduli modificati per effettuare un cambiamento

Manutenibilità (2)

- ◆ Alcuni attributi interni influiscono sulla manutenibilità e di conseguenza la loro misura può essere utile per caratterizzare il livello di manutenibilità del prodotto
 - Livello di strutturazione
 - Livello di documentazione
 - “Complessità”
- ◆ Più che per misurare il livello di manutenibilità, tali misure possono essere utilizzate per individuare eventuali aspetti/moduli critici

La misura degli attributi delle risorse

◆ Quali risorse?

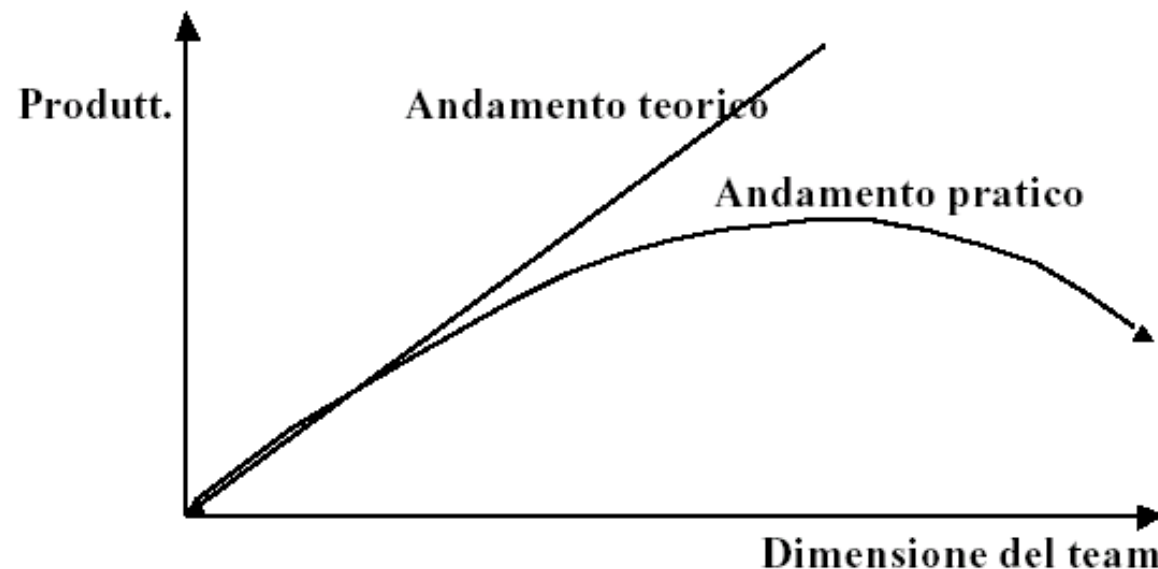
- Hardware
- Software
- Metodologie...
- ... ma soprattutto persone

◆ Il personale costituisce la risorsa fondamentale in un processo di sviluppo di software

◆ Il processo software è probabilmente il processo di sviluppo a più alta intensità umana

La produttività (1)

- ◆ In un processo ad alta intensità umana è fondamentale tenere sotto controllo la produttività del personale
- ◆ La nota legge di Brooks ci insegna che: “Aggiungere risorse ad un progetto in ritardo lo ritarda ulteriormente”



La produttività (2)

- ◆ Ma cosa intendiamo per produttività?
 - Se intendiamo il rapporto LOC/Effort la legge di Brooks non viene rispettata dalla maggior parte dei progetti
 - » Le persone producono codice (LOC) ma spesso producono codice “inutile” e di bassa qualità
 - La produttività deve essere più correttamente misurata in funzione delle “funzionalità” effettivamente sviluppate e della “qualità” del codice sviluppato

La produttività (3)

- ◆ Misure di produttività:
 - Dimensione/Sforzo
 - Punti funzione/Sforzo
 - Affidabilità/Sforzo

Altre misure relative al personale (1)

◆ Struttura del team

- Ha un impatto sulla facilità di comunicazione
- Si può misurare costruendo un grafo della comunicazione
 - » Nodi = persone
 - » Archi = Necessità di comunicazione
- A partire da tale grafo possiamo calcolare:
 - » Dimensione (numero di nodi)
 - » Densità di comunicazione (rapporto archi/nodi)
 - » Livello di comunicazione (tree impurity del grafo)
 - » Livello di comunicazione individuale (fan-in + fan-out dei singoli nodi)
 - » Livello di comunicazione medio (media del valore precedente)

Altre misure relative al personale (2)

- ◆ Esperienza con le tecnologie impiegate:
 - Può essere misurata secondo una classificazione (scala ordinale)
 - Nessuna esperienza
 - Conoscenza ma nessuna esperienza di lavoro
 - Esperienza in un solo progetto precedente
 - Esperienza in numerosi progetti precedenti
- ◆ Oppure come numero di giorni di training fatti

Alcune note finali

- ◆ Non chiedersi mai: “Quale misura devo effettuare”?
- ◆ Ma solo: “Quale entità voglio misurare”? “Quali attributi voglio misurare”? “Per quali obiettivi”?
- ◆ Non fidarsi di un “numero magico” ma definire con rigore l’intero processo di misura
- ◆ Non cercare di misurare ciò che non è misurabile
 - Accontentarsi di indicatori magari meno “affascinanti” ma più rigorosi